

Round 6 — adversarially review the worktree drawer and its new admission route

You have GitHub read access to this repo (`tom2025b/git-vista` , public). Everything below lives on the branch `feature/m11.03-worktree-drawer` (open as PR #654), NOT on `main` . Fetch the files yourself at that ref — do not ask to have them pasted, and do not guess at their contents.

Before answering anything, confirm you can actually read the branch: quote the exact sentence `Serviceable::refusal` returns for the `OutsideAllowedRoots` variant (`crates/git-vista-protocol/src/worktree.rs`). It is one sentence and it does not exist on `main` . If you cannot find it, say so and stop — a prior round produced a plausible-sounding but worthless answer built from a prompt alone after its source files failed to upload.

What this change does, in one paragraph

The previous slice (#547, merged as #651) taught the app to refuse a `git checkout` of a branch that is already checked out in another linked worktree, and to name the worktree holding it. That refusal offered *"open that worktree instead"* — and the offer failed, because it went to `POST /api/select` , which resolves ids through a server-owned catalog, and a linked worktree nobody ever scanned is not in the catalog. This PR adds `POST /api/select-worktree` , whose authority is a fresh `git worktree list --porcelain` census of the currently served repository rather than the catalog, and which **admits** the discovered worktree to the catalog before selecting it. It also adds a drawer listing every worktree.

Files to fetch

- `crates/git-vista-server/src/handlers/select.rs` — the new route `select_discovered_worktree` , and the `worktree_admission_tests` module at the bottom. This is the security-relevant file.

- `crates/git-vista-server/src/state.rs` — `register_discovered_worktree`, `path_is_allowed`, `allow_root`, `register_explicit`, `current`. The first of those is new; read it against the other four.
- `crates/git-vista-server/src/catalog.rs` — `Catalog::register`, `allow_root`, `contains_path`, `register_explicit`. Unchanged by this PR; it is the code the new route leans on, and the claim under review is a claim about it.
- `crates/git-vista-server/src/worktree_census.rs` — how `Serviceable` is computed, in particular the allowed-roots check and how a sibling's canonical path is derived. Unchanged by this PR.
- `crates/git-vista-protocol/src/worktree.rs` — `Serviceable`, its new `refusal()` and `is_openable()`, and `branch_holder`.
- `crates/git-vista/src/features/worktrees/core.rs` and `core_suite.rs` — the drawer's decision model and its tests, including the source-census tests that read the wasm-only view back.
- `crates/git-vista/src/features/worktrees/view.rs` — the markup. `cargo test` never compiles this file; that is why the source censuses exist.
- `docs/adr/0117-a-discovered-desk-needs-a-door-and-the-door-does-not-move-the-fence.md` — the argument being made. Judge the code against it, not the other way round.
- `docs/superpowers/specs/m3.23-worktrees.md` §1, the section headed "The security interaction, which is the part that needs care". This is the design the ADR claims to implement.

What the internal passes already covered — do not re-derive these

Spending your pass re-finding any of this is spending it on nothing:

1. **Three mutation arms, all caught**, at disjoint assertions in two crates: dropping refused rows from the drawer; making `register_discovered_worktree` call `allow_root`; collapsing the two `Serviceable` refusals into one badge reported as git's. The tests catch all three.

2. `./dev gate` **green end to end** — `fmt`, `clippy`, `wasm-clippy`, the full Rust suite (2952 tests, +24), a Trunk build, and 88 Playwright specs including five new ones that drive the drawer in a real browser.
3. **The route is classified** in `route_authz.rs` (`SessionAndCsrf`) and in the planner's write-route census as a catalog write rather than a git write.
4. **The happy path works end to end in a browser**, including switching to a worktree the catalog never held — that is the whole point of the change and it is covered.

What this round exists to answer

The claim under review is narrow and load-bearing. **The ADR asserts that admitting a discovered worktree can never widen the app's filesystem fence**, and that the guarantee is an *omission*:

`register_discovered_worktree` calls `Catalog::register` and deliberately does NOT call `allow_root` first, because doing so would make `git worktree add` a way to make the app serve any directory. The defence is claimed to be enforced twice — once by the census marking a sibling `Serviceable::Yes` only when its canonical path is already inside an allowed root, and again by `register` re-checking the roots itself and failing closed.

Please attack that specifically:

1. **Is the "enforced twice" claim actually true?** Read `Catalog::register` and confirm it re-checks the roots independently rather than trusting the caller. If the second check is weaker than the first, or is checking a different path than the one the first checked, say so.
2. **Is there a TOCTOU window?** The census canonicalises a path and tests it against the roots; `register` later canonicalises again. Between those two moments the path could change — a symlink swap, a directory replaced, a worktree moved. What is the worst thing an attacker with write access to the repository's own directory could do with that window? Note the fence check happens *inside* `register`, after its own canonicalisation, which is the reason I believe this is safe — check that reasoning rather than accepting it.

3. **Can a forged or crafted `worktree` id reach the registration step?** The id is opaque and must equal an id the census derived. Trace whether anything else can satisfy that comparison.
4. **The internal census runs with `expose_paths: true`.** The claim is that this is not a disclosure because nothing from that census is serialized to the client. Verify it. If any path can escape through an error message, a log line that reaches a response body, or a serialized field, that is a real finding.
5. **`read_only` is inherited from the currently served repository.** Is that the right provenance for a linked worktree, and can it be wrong in a way that grants write access to something that should be view-only?
6. **Are the source-census tests vacuous?** Several tests here assert things about source text rather than behaviour — including the one carrying the whole fence guarantee (`admitting_a_discovered_worktree_never_widens_the_allowed_roots`). A test that reads source can be trivially satisfied. Tell me if any of them would pass against code that had the defect they name.
7. **Anything in `view.rs` that decides something.** The file is supposed to decide nothing; `cargo test` never compiles it, so a decision hidden there is unreachable by every test in the repository.

What I am not asking for

Style, naming, comment density, or "you could also..." suggestions. Doc comments in this repo are deliberately long and carry the reasoning; that is a house convention, not an oversight. If you find nothing on the questions above, say so plainly rather than producing findings to fill the space — a clean answer to question 2 is worth more to me than five stylistic notes.

Signed: max · 2026-09-04T08:55:00-04:00